



Cairo University
Faculty of Computers and Artificial Intelligence

CS214-Data Structures

Dr. Basheer Adel Fatah

Assignment 2

Second Level

Second Semester

May 2026

Name	ID	Email
Rawan Youssry Ebrahim	20242130	20242130@stud.fci.cu.edu.eg
Doaa Hany AbdulRahman	20240180	20240180@stud.fci.cu.edu.eg
Abdulrahman Mostafa Abdelsamea	20240316	20240316@stud.fci.cu.edu.eg
Mohamed Ahmed Mohamed	20242267	20242267@stud.fci.cu.edu.eg

Part 4: Comparison:

BST (Sorted IDs):

```
5. Find books in specific range
6.Exit
Enter choice: 1
Enter Book ID: 23
Enter Book Title: w
Enter Author Name: w
Book added
Added Successfully
-----

1. Add Book
2. Search Book
3. Delete Book
4. Show All Books
5. Find books in specific range
6.Exit
Enter choice: 2
Enter ID to search: 23
The book is found on address 0000027B35BFA1A0
23 | w | w
It took 23 steps to find it
-----

1. Add Book
2. Search Book
3. Delete Book
4. Show All Books
5. Find books in specific range
6.Exit
Enter choice: 6
Program ended
The height of tree is 23
D:\DS_Ass2\ASS2\x64\Debug\ASS2.exe (process 24492) exited with code 0 (0x0).
Press any key to close this window . . .|
```

BST (Random IDs):

```
1000

642 fr r
656 se s
679 ws w
787 de e
987 e e
999 das ds
-----

1. Add Book
2. Search Book
3. Delete Book
4. Show All Books
5. Find books in specific range
6.Exit
Enter choice: 2
Enter ID to search: 666
Not found
Closest Book is found on address result: 656 | se | s
It took 11 steps to find it
-----

1. Add Book
2. Search Book
3. Delete Book
4. Show All Books
5. Find books in specific range
6.Exit
Enter choice: 6
Program ended
The height of tree is 11
D:\DS_Ass2\ASS2\x64\Debug\ASS2.exe (process 8472) exited with code 0 (0x0).
Press any key to close this window . . .|
```

AVL (sorted IDs):

```
enter id of the book  book you want to insert:20
enter the title: d
enter the author: s
Added Successfully
-----
Enter your choice: 1
enter id of the book  book you want to insert:21
enter the title: ds
enter the author: de
Added Successfully
-----
Enter your choice: 1
enter id of the book  book you want to insert:22
enter the title: dd
enter the author: d
Added Successfully
-----
Enter your choice: 1
enter id of the book  book you want to insert:23
enter the title: vf
enter the author: fv
Added Successfully
-----
Enter your choice: 3
Enter the id to search:
15
The book is found on address 00000289983C9A40
15 | g | f
It took 5 steps to find it
-----
Enter your choice: 6
Thanks for the usage
The height of tree is 5
D:\DS_Ass2\ASS2\x64\Debug\ASS2.exe (process 16688) exited with code 0 (0x0).
Press any key to close this window . . .|
```

AVL (random IDs):

```
enter id of the book  book you want to insert:256
enter the title: d
enter the author: d
Added Successfully
-----

Enter your choice: 1
enter id of the book  book you want to insert:556
enter the title: d
enter the author: sd
Added Successfully
-----

Enter your choice: 1
enter id of the book  book you want to insert:542
enter the title: w
enter the author: w
Added Successfully
-----

Enter your choice: 1
enter id of the book  book you want to insert:75
enter the title: d
enter the author: de
Added Successfully
-----

Enter your choice: 3
Enter the id to search:
10
The book is found on address 0000019A356CA7B0
10 | d | s
It took 4 steps to find it
-----

Enter your choice: 6
Thanks for the usage
The height of tree is 5
D:\DS_Ass2\ASS2\x64\Debug\ASS2.exe (process 21656) exited with code 0 (0x0).
Press any key to close this window . . .|
```

Performance Comparison and Observations

Based on the experimental results, the AVL Tree demonstrated superior performance compared to the BST, especially with sorted data. Here are the key findings:

Tree Height Efficiency: When inserting books with sorted IDs (e.g., from 20 to 30), the BST became "skewed," essentially turning into a linked list where the height was equal to the number of nodes ($H = N$). In contrast, the AVL Tree maintained a balanced structure through rotations, resulting in a much smaller height ($H \approx \log N$).

Search Optimization: The search operations in the AVL Tree were significantly faster, requiring fewer steps (comparisons) to locate a book. In the BST, the search steps increased linearly with the number of nodes, while the AVL kept the search complexity at a consistent logarithmic scale.

Conclusion: The AVL Tree is more efficient for real-world applications where data might be entered in a sorted or semi-sorted manner, as it guarantees optimal performance by preventing tree imbalance.

How Heap Works

A Heap is a complete binary tree commonly used to implement priority systems efficiently.

In this project, a Max Heap was used because the patient with the highest severity must always be at the top.

Heap Operations

1. Insert Patient

When a new patient is added:

- The patient is inserted at the end of the heap.
- Heapify Up is performed.
- The node moves upward until the heap property is restored.

Time Complexity:

$O(\log n)$

2. Treat Next Patient

The patient with the highest priority is removed from the root.

Steps:

- Remove the root patient.
- Replace it with the last element.
- Perform Heapify Down to restore heap order.

Time Complexity:

$O(\log n)$

3. View Next Patient

The top patient is displayed without removing them.

Time Complexity:

$O(1)$

4. Update Severity

If a patient's severity changes:

- The severity value is updated.
- Heapify Up or Heapify Down is applied depending on the new value.

Time Complexity:

$O(\log n)$

Why Heap Is Efficient

Heap is efficient because it keeps the highest priority element at the root. Insertion and deletion operations are performed in logarithmic time. This makes heap suitable for systems such as emergency rooms, CPU scheduling, task management systems, and network routing.

Comparison with Other Structures

Structure	Insert	Delete Highest Priority	Search	Notes
Heap	$O(\log n)$	$O(\log n)$	$O(n)$	Best for priority systems
Array	$O(1)$	$O(n)$	$O(n)$	Simple but inefficient
Linked List	$O(1)$	$O(n)$	$O(n)$	Requires traversal
BST	$O(\log n)$ avg	$O(\log n)$ avg	$O(\log n)$ avg	More complex
AVL Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	Balanced but harder

Manual Heap vs STL priority_queue

Manual Heap	STL priority_queue
Implemented from scratch	Built-in STL container
Helps understand heap internals	Easier and faster to use
Requires manual heapify operations	Automatically maintains heap
Better for learning data structures	Better for real applications

Time Complexity Comparison

Operation	Manual Heap	STL priority_queue
Insert	$O(\log n)$	$O(\log n)$
Remove Top	$O(\log n)$	$O(\log n)$
View Top	$O(1)$	$O(1)$
Ease of Use	Harder	Easier